Task 1:

1a My Answer:

Variable	Value displayed
result1	2
result2	2.33333

1b My Explanation:

These two calculations produce different results, despite both using a and b, because result1 does not specify the data type that should be used for the calculation so the result is stored as an integer as both values used (a and b) are both integers. This means that result1 can only be whole numbers so when the division is done the result is 2.3 recurring but as integer division is done the result is truncated to just 2. However, in result2 the data type is specified as a double for the value of a. This changes the data type of a when using in the division to be a double so the result is stored as a double rather than an integer as both data types in the division are not the same. By default cout only displays 6 significant figures so 2.33333 is shown preventing 3 from being printed indefinitely.

1c My Answer:

Information	C++ data type	Reason for your choice
A person's age	int	Age is a whole number.
The price of a product, such as £19.95	double	Can represent decimals. Has double the precision of float so is better for calculations.
A student's first initial	char	Can only represent one character so is perfect for first initial.
Whether a user is logged in	bool	True or false condition whether the user is logged in or not.

Task 2:

2a My Answer & Explanation:

The program will display “2:1”.

The condition “if (mark >= 60)” is run because it is the first condition in the if-else ladder that is true, as the variable “submitted” is false so that condition is skipped and the next condition is checked which is false as mark is less than 70 but the next condition checked is true as the mark 68 is greater than 60.

2b My Answer & Explanation:

The program will display “No submission”.

This is because the first condition that is checked is if “submitted” is false which in this case is true so the if-else ladder ends here and No submission is printed to the console. The value of the variable “mark” does not matter as no condition that checks the mark is executed before the if-else ladder is exited.

2c My Answer:

if (age >= 18 && age <= 120)

2d My Explanation:

A switch statement is clearer when there are many discrete values to choose from such as a main menu selector. This is because there is ont input value being compared against several fixed options.

Task 3:

3a My Answer:

Iteration	Value of i	Condition: true or false?	Value of total after iteration
1	1	false	-1
2	2	true	1
3	3	false	0
4	4	true	4
5	5	false	3

The final value is 3.

3b My Answer & Explanation:

In line 5 n++ is used which will increment n by 1 every iteration rather than decrease n by 1. It should be replaced with n—to decrease n by one every iteration so it counts backwards from the value of n to 0, rather than counting up.

3c My Answer:

```
int n = 5;



while (n >= 0) {
   cout << n << " ";
   n--;
}
```

3d My Explanation:

A for loop does something for a given range while a while loop runs forever until a given condition is true.

For loop use: Iterating through a list of ages and printing the largest.

While loop use: Input validation for a new password entered by a user.

Task 4:

4a My Answer & Explanation:

The value "51" is stored at "marks[2]". This is because in C++ arrays start at index 0. So, the first item in an array is stored at index 0, and the second is stored at index 1. From this I can see that "marks[2]" is referring to the item in the 3rd index position which is "51".

4b My Answer & Explanation:

All valid index positions in "marks[5]" are "0, 1, 2, 3, 4". Attempting to access "marks[5]" is incorrect because it doesn't exist as that is looking for the item in index 5 which doesn't exist as when the array is declared it is made with 5 index spaces which start from 0 rather than 1 so the index 5 does not exist as that would mean that marks have 6 spaces.

4c My Answer & Explanation:

The error is that in the for-loop condition where "i <= 5" is used. This causes index 5 to be printed but the array only has indexes 0 to 4 (inclusive) used, so 0 is printed. This can be fixed by either changing the "<=" to just a "<" or by changing the 5 to a 4 in the condition.

4d My Answer:

```
int sumOfMarks = 0

for (int i = 0; i <=4; i++) {

   sumOfMarks += marks[i];

}

cout << sumOfMarks << endl;
```

Task 5:

5a My Answer:

The two parameters of addBonus() are: “mark” and “bonus” and both are integers.

In main when addBonus() is called the value of “score” which is “64” is passed to “mark”, and the value of “extra” which is “5” is passed to the parameter “bonus”.

5b My Answer & Explanation:

addBonus() returns the value “69” when called by main().

This is calculated by adding the value of “mark” and “bonus” and storing the sum into the newly declared variable “finalMark” which is then returned to main().

5c My Answer & Explanation:

Variable	Function in which it is declared	Initial value, if explicitly assigned
score	main()	64
extra	main()	5
mark	addBonus()	64 (value of score)
bonus	addBonus()	5 (value of extra)
finalMark	addBonus()	69 (sum of mark + bonus)

main() cannot directly access the variable “finalMark” as it is local to addBonus() and not shared with main().

5d My Answer & Explanation:

1. The values “15” and “5” are displayed by the program.
2. The value of “number” in main() does not change after calling increase() because it increase() does not directly access nor change the variable “number”. “number” is passed into increase() as an argument so the value is passed into the function rather than modifying the variable itself.

5e My Answer & Explanation:

One advantage of using functions to divide a large program into smaller functions is that it allows the reuse of blocks of code so the same chunks of code aren’t being rewritten many times.

Another advantage is that use of smaller functions makes the code easier to read, debug, understand, and modify certain parts of the code without affecting other parts of the program.

Task 6:

6a My Answer:

```
choice = 0


WHILE choice != 4 DO
   OUTPUT “SIMPLE NUMBER CALCULATOR”
   OUTPUT “1. Add two numbers”
   OUTPUT “2. Subtract two numbers”
   OUTPUT “3. Multiply two numbers”
   OUTPUT “4. Exit”
   OUTPUT “”
   OUTPUT “Enter your choice:”
   INPUT choice


   IF choice = 1 THEN
      OUTPUT “Enter first number:”
      INPUT num1
      OUTPUT “Enter second number:”
      INPUT num2
```

```
        result = num1 + num2
        OUTPUT “Result: ”, result

    ELSE IF choice = 2 THEN
        OUTPUT “Enter first number:”
        INPUT num1
        OUTPUT “Enter second number:”
        INPUT num2
        result = num1 - num2
        OUTPUT “Result: ”, result

    ELSE IF choice = 3 THEN
        OUTPUT “Enter first number:”
        INPUT num1
        OUTPUT “Enter second number:”
        INPUT num2
        result = num1 * num2
        OUTPUT “Result: ”, result

    ELSE IF choice = 4 THEN
        OUTPUT "Exiting calculator.”

    ELSE
        OUTPUT “Invalid choice. Please enter 1, 2, 3 or 4.”
    END IF

END WHILE
```

6b My Answer:

Function name	Purpose
displayMenu()	Display the calculator menu.
getMenuChoice()	Read the user's input choice and checks if valid.
performCalculation()	Perform the calculation entered.

6c My Explanation:

If the user enters "7" when prompted to choose an option from the menu, then the program should recognise that this is an invalid choice as it is not a number between 1 and 4. The program should handle this by recognising that this is an invalid input and show the menu again whilst asking the user to enter a valid input. This can be done with either a default case if a switch is used or by and else condition if an else-if ladder is used.

Task 7:

7a My Answer & Explanation:

Problem	What is wrong?	Why is it a problem?
1	sum = sum + num	This is a problem because it adds both positive and negative numbers to sum rather than only adding positive numbers.
2	count = count + 1	It increments whether the value is positive or negative when it should only increment where there is a positive value.
3	-1 is processed before the value is checked if is it positive or negative	This is an issue as we only want to add positive numbers and not negative numbers.
4	Return sum / count	The return statement uses integer division so if there is a decimal it is ignored by the program. It should use static_cast<double>().

7b My Answer:

```
double averagePositive() {
    double sum = 0.0;
    int count = 0;
    int num = 0;

    while (true) {
        cin >> num;

        if (num == -1) {
            break;
        }

        if (num > 0) {
            sum = sum + num;
            count = count + 1;
        }
    }

    if (count == 0) {
        return 0.0;
    }

    return sum / count;
}
```

7c My Explanation:

Sum is now declared as a double rather than and integer so when the division is done in the return calculation it will show decimal places rather than truncating the decimal.

The first thing that is executed in the do while loop is to check if -1 has been entered. If it has been then the input is stopped, otherwise if it is not the first thing then the program will process the -1 before calculating the average.

There is now a check to see whether the number entered is larger than 0 before adding the number to the sum and incrementing the count, as it only accepts positive integers larger than 0.

Task 8:

8a My Answer & Explanation:

Test	Input values	Expected result	Reason for test
1	10, 20, 30, -1	20.0	Normal input with several positive numbers. Checks the main intended behaviour.
2	0, -6, -7, -1	0.0	No positive numbers are entered. Checks that zero and negatives are ignored and that division by zero is avoided.
3	4, -1	4.0	Single positive integer, checks the average of one value is the value itself
4	5, 0, -4, 15, -20, -1	10.0	Mixture of positives, zero and negatives. Checks that only 5 and 15 are included.
5	-1	0.0	-1 is entered immediately. Checks that input stops correctly and the function returns 0.0 when no positive numbers are entered.

8b My Answer:

If a program is only tested with one normal example is not sufficient as it will only show that the code works with one expected case. Edge cases like zero, maximum/minimum values, and empty inputs must be tested as well as they can often lead to overflow errors or program crashes. Invalid inputs should be used to test whether the program validates data and handles errors safely instead of crashing or producing wrong results. Preventing the risk of program failures in front of the user.

AI Declaration

Complete the table below and attach it as an appendix to your submission. This table is not included in any submission word count. Use it to **declare** your level of generative AI usage for your submission by selecting one out of **Solo**, **Assisted** or **Partnered Work**. If you select **Assisted Work**, indicate how AI was used.

Solo Work	**S1 – Generative AI tools have not been used for the preparation of your submission.**	☒
Assisted Work	**Generative AI tools have been used to assist you to prepare your submission.** Please indicate below the general tasks for which you have used AI. Select all that apply.	☐
	A1 – Idea Generation and Problem Exploration Used to generate ideas, and to explore approaches to solving a problem, developing a design, or investigating a question. You must critically assess AI-generated suggestions and ensure your own intellectual contributions are central.	☐
	A2 – Planning and Structuring Reports or Projects Used to help outline the structure of reports, documentation and projects. The final structure and implementation must be your own work.	☐
	A3 – Research Assistance Used to locate and summarise relevant articles, technical documentation or online resources. The interpretation and integration of research into your submission remain your responsibility.	☐
	A4 – Language Refinement Used to check grammar, refine language or improve sentence structure in reports. AI should be used only to suggest improvements, and you should check the original meaning is preserved. You must ensure that code documentation accurately reflects the code and is correct.	☐
	A5 – Mathematical Calculations Used to suggest appropriate mathematical methods or to check solutions. AI should not be used to produce the mathematical workings and descriptions that you submit.	☐
	A6 – Solution Architecture and Design Used to help outline the architecture or structure of a technical solution, such as system components, class hierarchies, mathematical models, engineering designs or processes. The final design and structure must be your own.	☐
	A7 – Coding and Computational Assistance Used to generate example code, understand syntax, explore alternative implementation, use computational tools, or learn new coding or computational concepts. Any submitted code must be your own, and you must be able to explain it.	☐
	A8 – Review and Improvement of Technical Work Used to review code, calculations, models, designs, simulations, or other technical work; improve readability, structure or presentation; or help correct errors. AI should be used only to provide suggestions for improvement. You must ensure the output accurately reflects your knowledge.	☐
	A9 – Testing, Verification and Validation Support Used to create test cases, validate outputs or suggest edge cases for software testing, or guide verification and validation. You are responsible for test planning and interpreting results.	☐
	A10 – Data Analysis and Visualisation Used to suggest ways to analyse, present or visualise datasets. The analysis and interpretations that you present must be your own.	☐
	A11 – Other AI roles not listed above Please specify:	☐
Partnered Work	**P1 – Generative AI tool use forms an integral part of the assessment and is compliant with the assessment brief.**	☐

By completing this table, you are making a declaration about how you have used AI to prepare your assessment. Please note that the above uses can and will change as AI tools and technologies change. We reserve the right to update the form at the end of each academic year.